Section 8.2 - Baseline vs Image-Augmented Convnets

November 26, 2025

1 A baseline convnet from scratch

1.1 Dataset

1) Dog-vs-cat classification: 1 = dog & 0 = cat
2) Balanced dataset: 2.000 Training + 1.000 Validation + 2.000 Testing
3) Data processing pipeline: image_dataset_from_directory()

- Read JPEG picture files
- Decode JPEG content to RGB grids of pixels
- Convert into floating-point tensors
- Resize to 180 x 180 x 3
- Pack in batches

[1]:
```
from pathlib import Path
from tensorflow.keras.utils import image_dataset_from_directory

new_base_dir = Path("cats_vs_dogs_small")

train_dataset = image_dataset_from_directory(
    new_base_dir / "train",
    image_size=(180, 180),
    batch_size=32)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation",
    image_size=(180, 180),
    batch_size=32)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test",
    image_size=(180, 180),
    batch_size=32)
```

```
Found 2000 files belonging to 2 classes.
Found 1000 files belonging to 2 classes.
Found 2000 files belonging to 2 classes.
```

[2]:
```
# check the shapes in the dataset
for data_batch, labels_batch in train_dataset:
```

```
    print("data batch shape:", data_batch.shape)
    print("labels batch shape:", labels_batch.shape)
    break
```

```
data batch shape: (32, 180, 180, 3)
labels batch shape: (32,)
```

1.2 Model architecture

- Model: a stack of Conv2D & MaxPooling2D layers
- Depth of feature maps increases in the model
- Size of feature maps decreases in the model
- Normalization: [0, 255] –> [0, 1]
- End with a Dense layer with units = 1 & sigmoid activation
- Larger images, more complex problems –> more layers: augment capacity
- Feature map not overly large at the Flatten layer

[3]:
```
from tensorflow import keras
from tensorflow.keras import layers

inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1./255)(inputs)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)

# inspect the model
model.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 180, 180, 3)]     0

 rescaling (Rescaling)       (None, 180, 180, 3)       0

 conv2d (Conv2D)             (None, 178, 178, 32)      896

 max_pooling2d (MaxPooling2D  (None, 89, 89, 32)       0
```

```
)

conv2d_1 (Conv2D)            (None, 87, 87, 64)        18496

max_pooling2d_1 (MaxPooling  (None, 43, 43, 64)        0
2D)

conv2d_2 (Conv2D)            (None, 41, 41, 128)       73856

max_pooling2d_2 (MaxPooling  (None, 20, 20, 128)       0
2D)

conv2d_3 (Conv2D)            (None, 18, 18, 256)       295168

max_pooling2d_3 (MaxPooling  (None, 9, 9, 256)         0
2D)

conv2d_4 (Conv2D)            (None, 7, 7, 256)         590080

flatten (Flatten)            (None, 12544)             0

dense (Dense)                (None, 1)                 12545

=================================================================
Total params: 991,041
Trainable params: 991,041
Non-trainable params: 0
_________________________________________________________________
```

1.3 Training

- RMSprop optimizer + "binary crossentropy" loss + "accuracy" metrics
- Callbacks: ModelCheckPoint with save_best_only & "val_loss" monitor

[4]:
```
model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_from_scratch.h5",
        save_best_only=True,
        monitor="val_loss")
]

history = model.fit(
    train_dataset,
    epochs=5,
```

```
    validation_data=validation_dataset,
    callbacks=callbacks)
```

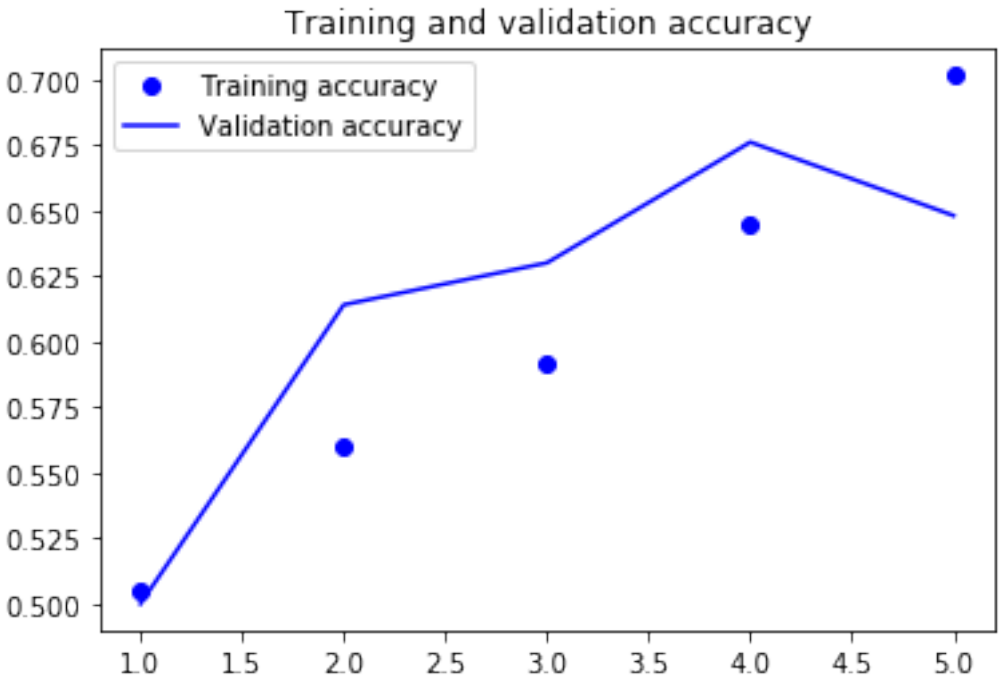
```
Epoch 1/5
63/63 [==============================] - 4s 41ms/step - loss: 0.7551 - accuracy:
0.5055 - val_loss: 0.7978 - val_accuracy: 0.5000
Epoch 2/5
63/63 [==============================] - 2s 31ms/step - loss: 0.6935 - accuracy:
0.5600 - val_loss: 0.6700 - val_accuracy: 0.6140
Epoch 3/5
63/63 [==============================] - 2s 31ms/step - loss: 0.6798 - accuracy:
0.5920 - val_loss: 0.6288 - val_accuracy: 0.6300
Epoch 4/5
63/63 [==============================] - 2s 31ms/step - loss: 0.6281 - accuracy:
0.6450 - val_loss: 0.5895 - val_accuracy: 0.6760
Epoch 5/5
63/63 [==============================] - 2s 31ms/step - loss: 0.5755 - accuracy:
0.7015 - val_loss: 0.6514 - val_accuracy: 0.6480
```

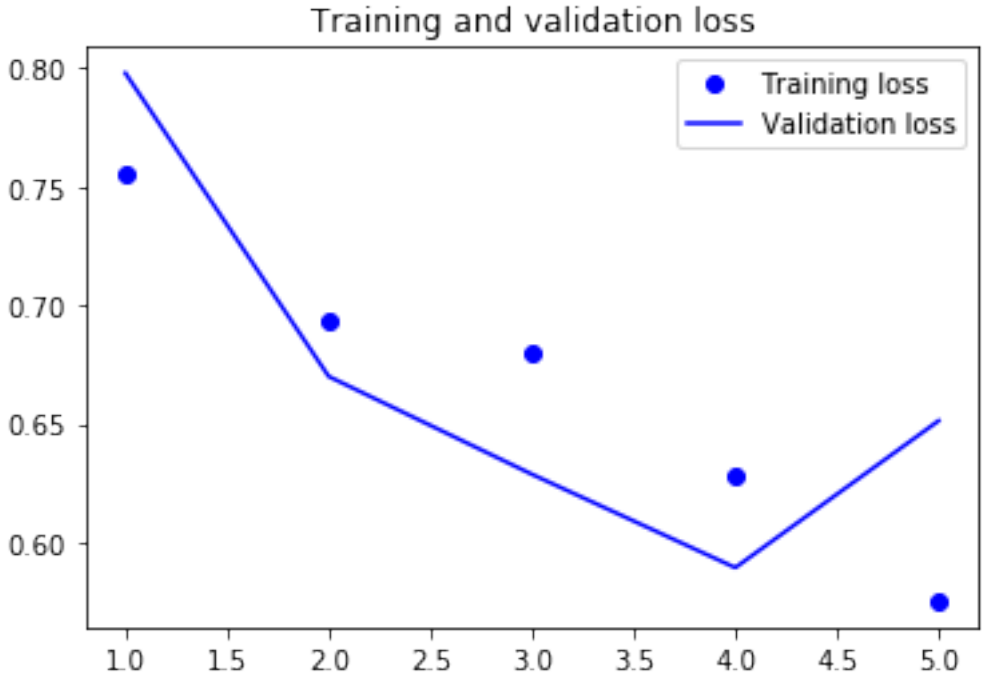
1.4 Plotting the training curves

- Overfitting appears after 4 epochs

[5]:
```
import matplotlib.pyplot as plt

accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```


Training and validation accuracy
Training accuracy
Validation accuracy
0.700
0.675
0.650
0.625
0.600
0.575
0.550
0.525
0.500
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0


Training and validation loss
Training loss
Validation loss
0.80
0.75
0.70
0.65
0.60
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

1.5 Evaluation on the test dataset

- Accuracy: about 68%

[6]:
```
test_model = keras.models.load_model("convnet_from_scratch.h5")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

```
63/63 [==============================] - 1s 11ms/step - loss: 0.6040 - accuracy:
0.6820
Test accuracy: 0.682
```

2 Data augmentation

1) Purpose: Mitigate overfitting by adding useful representations
2) Random transformations: Generate believable looking images

- RandomFlip "horizontal"
- RandomRotation [-36, 36] degrees
- RandomZoom [-20%, 20%]

[7]:
```
# Define a data augmentation stage
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.2),
    ]
)
```

[11]:
```
# Hide the warning logs messages
import tensorflow as tf
import logging

tf.get_logger().setLevel(logging.ERROR)

# Inspect some randomly augmented images
plt.figure(figsize=(10, 10))
for images, _ in train_dataset.take(1):
    for i in range(9):
        augmented = data_augmentation(images)
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(augmented[0].numpy().astype("uint8"))
        plt.axis("off")
```

2.1 Convnet with image augmentation & dropout

[15]:
```
inputs = keras.Input(shape=(180, 180, 3))
x = data_augmentation(inputs)
x = layers.Rescaling(1./255)(x)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
```

```
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model_image_augmented = keras.Model(inputs=inputs, outputs=outputs, name =␣
 ↪"model_image_augmented")

# inspect the model
model_image_augmented.summary()
```

```
Model: "model_image_augmented"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_5 (InputLayer)        [(None, 180, 180, 3)]     0

 sequential (Sequential)     (None, 180, 180, 3)       0

 rescaling_4 (Rescaling)     (None, 180, 180, 3)       0

 conv2d_20 (Conv2D)          (None, 178, 178, 32)      896

 max_pooling2d_16 (MaxPoolin  (None, 89, 89, 32)       0
 g2D)

 conv2d_21 (Conv2D)          (None, 87, 87, 64)        18496

 max_pooling2d_17 (MaxPoolin  (None, 43, 43, 64)       0
 g2D)

 conv2d_22 (Conv2D)          (None, 41, 41, 128)       73856

 max_pooling2d_18 (MaxPoolin  (None, 20, 20, 128)      0
 g2D)

 conv2d_23 (Conv2D)          (None, 18, 18, 256)       295168

 max_pooling2d_19 (MaxPoolin  (None, 9, 9, 256)        0
 g2D)

 conv2d_24 (Conv2D)          (None, 7, 7, 256)         590080

 flatten_4 (Flatten)         (None, 12544)             0

 dropout_3 (Dropout)         (None, 12544)             0

 dense_4 (Dense)             (None, 1)                 12545
```

```
=================================================================
Total params: 991,041
Trainable params: 991,041
Non-trainable params: 0
_________________________________________________________________
```

2.2 Training

- RMSprop optimizer + "binary crossentropy" loss + "accuracy" metrics
- Callbacks: ModelCheckPoint with save_best_only & "val_loss" monitor
- 100 epochs - without image augmentation: overfitting after 4 epochs

[17]:
```
model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_image_augmented.h5",
        save_best_only=True,
        monitor="val_loss")
]

history = model.fit(
    train_dataset,
    epochs=100,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

```
Epoch 1/100
63/63 [==============================] - 8s 76ms/step - loss: 0.7226 - accuracy:
0.5200 - val_loss: 0.6880 - val_accuracy: 0.5020
Epoch 2/100
63/63 [==============================] - 5s 75ms/step - loss: 0.6876 - accuracy:
0.5490 - val_loss: 0.6675 - val_accuracy: 0.6010
Epoch 3/100
63/63 [==============================] - 5s 75ms/step - loss: 0.6779 - accuracy:
0.6100 - val_loss: 0.7574 - val_accuracy: 0.5710
Epoch 4/100
63/63 [==============================] - 5s 75ms/step - loss: 0.6528 - accuracy:
0.6265 - val_loss: 0.6618 - val_accuracy: 0.5900
Epoch 5/100
63/63 [==============================] - 5s 73ms/step - loss: 0.6384 - accuracy:
0.6535 - val_loss: 0.8433 - val_accuracy: 0.5730
Epoch 6/100
63/63 [==============================] - 5s 74ms/step - loss: 0.6222 - accuracy:
0.6745 - val_loss: 0.5842 - val_accuracy: 0.6950
Epoch 7/100
```

```
63/63 [==============================] - 5s 74ms/step - loss: 0.6081 - accuracy:
0.6915 - val_loss: 0.6292 - val_accuracy: 0.6370
Epoch 8/100
63/63 [==============================] - 5s 74ms/step - loss: 0.5740 - accuracy:
0.7085 - val_loss: 0.6199 - val_accuracy: 0.6880
Epoch 9/100
63/63 [==============================] - 5s 74ms/step - loss: 0.5672 - accuracy:
0.7100 - val_loss: 0.6731 - val_accuracy: 0.6290
Epoch 10/100
63/63 [==============================] - 5s 76ms/step - loss: 0.5559 - accuracy:
0.7290 - val_loss: 0.5632 - val_accuracy: 0.7000
Epoch 11/100
63/63 [==============================] - 5s 74ms/step - loss: 0.5538 - accuracy:
0.7310 - val_loss: 0.5801 - val_accuracy: 0.6940
Epoch 12/100
63/63 [==============================] - 5s 75ms/step - loss: 0.5180 - accuracy:
0.7450 - val_loss: 0.5312 - val_accuracy: 0.7450
Epoch 13/100
63/63 [==============================] - 5s 74ms/step - loss: 0.5141 - accuracy:
0.7435 - val_loss: 0.5639 - val_accuracy: 0.7080
Epoch 14/100
63/63 [==============================] - 5s 73ms/step - loss: 0.4951 - accuracy:
0.7615 - val_loss: 0.8829 - val_accuracy: 0.6190
Epoch 15/100
63/63 [==============================] - 5s 74ms/step - loss: 0.5066 - accuracy:
0.7620 - val_loss: 0.5162 - val_accuracy: 0.7550
Epoch 16/100
63/63 [==============================] - 5s 74ms/step - loss: 0.4931 - accuracy:
0.7660 - val_loss: 0.5679 - val_accuracy: 0.7240
Epoch 17/100
63/63 [==============================] - 5s 75ms/step - loss: 0.4845 - accuracy:
0.7790 - val_loss: 0.6046 - val_accuracy: 0.7140
Epoch 18/100
63/63 [==============================] - 5s 74ms/step - loss: 0.4690 - accuracy:
0.7765 - val_loss: 0.5180 - val_accuracy: 0.7590
Epoch 19/100
63/63 [==============================] - 5s 73ms/step - loss: 0.4334 - accuracy:
0.8015 - val_loss: 0.5555 - val_accuracy: 0.7630
Epoch 20/100
63/63 [==============================] - 5s 75ms/step - loss: 0.4302 - accuracy:
0.7980 - val_loss: 0.5307 - val_accuracy: 0.7660
Epoch 21/100
63/63 [==============================] - 5s 73ms/step - loss: 0.4103 - accuracy:
0.8060 - val_loss: 0.5614 - val_accuracy: 0.7340
Epoch 22/100
63/63 [==============================] - 5s 73ms/step - loss: 0.4045 - accuracy:
0.8240 - val_loss: 0.5655 - val_accuracy: 0.7620
Epoch 23/100
```

```
63/63 [==============================] - 5s 74ms/step - loss: 0.4023 - accuracy:
0.8220 - val_loss: 0.5378 - val_accuracy: 0.7840
Epoch 24/100
63/63 [==============================] - 5s 75ms/step - loss: 0.4004 - accuracy:
0.8145 - val_loss: 0.4987 - val_accuracy: 0.7950
Epoch 25/100
63/63 [==============================] - 5s 75ms/step - loss: 0.3715 - accuracy:
0.8280 - val_loss: 0.6165 - val_accuracy: 0.7480
Epoch 26/100
63/63 [==============================] - 5s 74ms/step - loss: 0.3788 - accuracy:
0.8300 - val_loss: 0.5180 - val_accuracy: 0.8050
Epoch 27/100
63/63 [==============================] - 5s 73ms/step - loss: 0.3341 - accuracy:
0.8625 - val_loss: 0.5199 - val_accuracy: 0.7970
Epoch 28/100
63/63 [==============================] - 5s 73ms/step - loss: 0.3301 - accuracy:
0.8635 - val_loss: 0.5859 - val_accuracy: 0.7750
Epoch 29/100
63/63 [==============================] - 5s 74ms/step - loss: 0.3505 - accuracy:
0.8500 - val_loss: 0.5732 - val_accuracy: 0.7890
Epoch 30/100
63/63 [==============================] - 5s 73ms/step - loss: 0.3303 - accuracy:
0.8570 - val_loss: 0.5150 - val_accuracy: 0.8030
Epoch 31/100
63/63 [==============================] - 5s 73ms/step - loss: 0.3016 - accuracy:
0.8690 - val_loss: 0.5213 - val_accuracy: 0.7990
Epoch 32/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2881 - accuracy:
0.8745 - val_loss: 0.6005 - val_accuracy: 0.8040
Epoch 33/100
63/63 [==============================] - 5s 75ms/step - loss: 0.3165 - accuracy:
0.8780 - val_loss: 0.6627 - val_accuracy: 0.7820
Epoch 34/100
63/63 [==============================] - 5s 75ms/step - loss: 0.2813 - accuracy:
0.8830 - val_loss: 0.6057 - val_accuracy: 0.7770
Epoch 35/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2885 - accuracy:
0.8810 - val_loss: 0.5452 - val_accuracy: 0.7930
Epoch 36/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2679 - accuracy:
0.8805 - val_loss: 0.7449 - val_accuracy: 0.7630
Epoch 37/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2557 - accuracy:
0.8940 - val_loss: 0.6757 - val_accuracy: 0.8010
Epoch 38/100
63/63 [==============================] - 5s 75ms/step - loss: 0.2638 - accuracy:
0.8925 - val_loss: 0.7220 - val_accuracy: 0.8040
Epoch 39/100
```

```
63/63 [==============================] - 5s 75ms/step - loss: 0.2606 - accuracy:
0.8980 - val_loss: 0.5675 - val_accuracy: 0.7960
Epoch 40/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2537 - accuracy:
0.9020 - val_loss: 0.6553 - val_accuracy: 0.7710
Epoch 41/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2319 - accuracy:
0.9050 - val_loss: 0.7104 - val_accuracy: 0.7940
Epoch 42/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2225 - accuracy:
0.9060 - val_loss: 0.5622 - val_accuracy: 0.8090
Epoch 43/100
63/63 [==============================] - 5s 75ms/step - loss: 0.2233 - accuracy:
0.9125 - val_loss: 0.5714 - val_accuracy: 0.8070
Epoch 44/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1888 - accuracy:
0.9265 - val_loss: 0.8425 - val_accuracy: 0.7880
Epoch 45/100
63/63 [==============================] - 5s 73ms/step - loss: 0.2193 - accuracy:
0.9165 - val_loss: 0.8104 - val_accuracy: 0.7790
Epoch 46/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2087 - accuracy:
0.9185 - val_loss: 0.8201 - val_accuracy: 0.7900
Epoch 47/100
63/63 [==============================] - 5s 74ms/step - loss: 0.2175 - accuracy:
0.9165 - val_loss: 0.6922 - val_accuracy: 0.7830
Epoch 48/100
63/63 [==============================] - 5s 73ms/step - loss: 0.1903 - accuracy:
0.9250 - val_loss: 0.7908 - val_accuracy: 0.8010
Epoch 49/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1974 - accuracy:
0.9270 - val_loss: 0.9830 - val_accuracy: 0.7800
Epoch 50/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1859 - accuracy:
0.9290 - val_loss: 0.7487 - val_accuracy: 0.7940
Epoch 51/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1712 - accuracy:
0.9330 - val_loss: 1.1191 - val_accuracy: 0.7650
Epoch 52/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1747 - accuracy:
0.9350 - val_loss: 0.8018 - val_accuracy: 0.7930
Epoch 53/100
63/63 [==============================] - 5s 73ms/step - loss: 0.1737 - accuracy:
0.9350 - val_loss: 0.7580 - val_accuracy: 0.8030
Epoch 54/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1636 - accuracy:
0.9380 - val_loss: 0.8598 - val_accuracy: 0.7990
Epoch 55/100
```

```
63/63 [==============================] - 5s 74ms/step - loss: 0.1964 - accuracy:
0.9345 - val_loss: 0.9979 - val_accuracy: 0.7580
Epoch 56/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1822 - accuracy:
0.9395 - val_loss: 0.8693 - val_accuracy: 0.7960
Epoch 57/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1819 - accuracy:
0.9405 - val_loss: 0.7238 - val_accuracy: 0.8300
Epoch 58/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1762 - accuracy:
0.9370 - val_loss: 0.9068 - val_accuracy: 0.7950
Epoch 59/100
63/63 [==============================] - 5s 79ms/step - loss: 0.1709 - accuracy:
0.9395 - val_loss: 0.7617 - val_accuracy: 0.8170
Epoch 60/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1619 - accuracy:
0.9425 - val_loss: 0.7365 - val_accuracy: 0.7880
Epoch 61/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1605 - accuracy:
0.9455 - val_loss: 1.0749 - val_accuracy: 0.7800
Epoch 62/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1701 - accuracy:
0.9430 - val_loss: 0.9667 - val_accuracy: 0.8150
Epoch 63/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1735 - accuracy:
0.9350 - val_loss: 1.1734 - val_accuracy: 0.7640
Epoch 64/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1382 - accuracy:
0.9530 - val_loss: 1.1555 - val_accuracy: 0.7680
Epoch 65/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1726 - accuracy:
0.9430 - val_loss: 0.9993 - val_accuracy: 0.7900
Epoch 66/100
63/63 [==============================] - 5s 73ms/step - loss: 0.1792 - accuracy:
0.9400 - val_loss: 1.0454 - val_accuracy: 0.7610
Epoch 67/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1425 - accuracy:
0.9430 - val_loss: 0.7893 - val_accuracy: 0.8040
Epoch 68/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1408 - accuracy:
0.9490 - val_loss: 0.8993 - val_accuracy: 0.7970
Epoch 69/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1746 - accuracy:
0.9385 - val_loss: 0.9261 - val_accuracy: 0.7770
Epoch 70/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1368 - accuracy:
0.9490 - val_loss: 0.9758 - val_accuracy: 0.7820
```

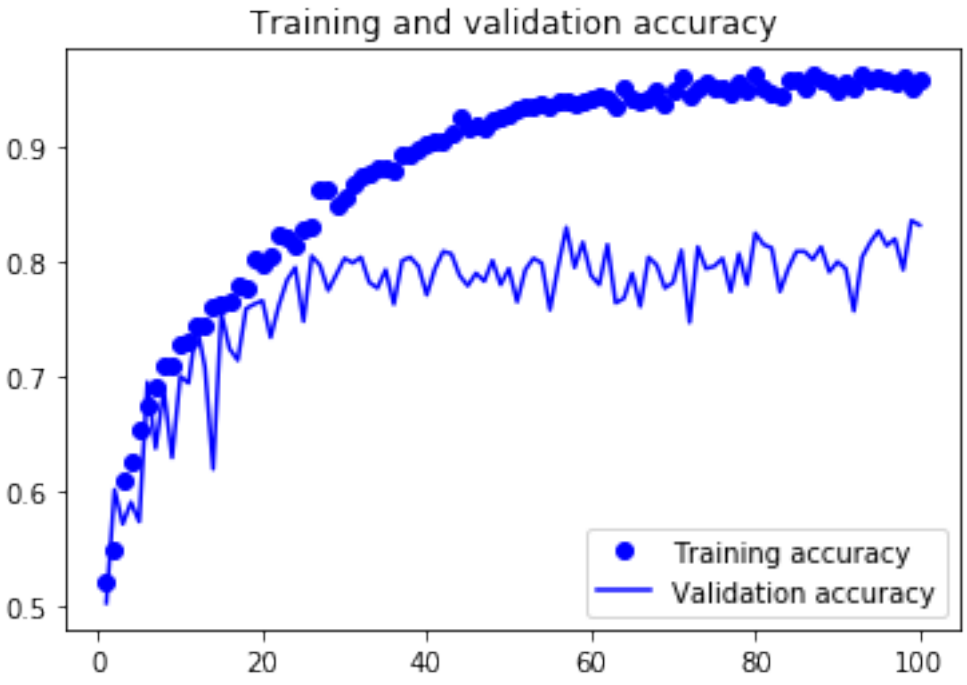
```
Epoch 71/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1035 - accuracy:
0.9620 - val_loss: 0.9785 - val_accuracy: 0.8100
Epoch 72/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1632 - accuracy:
0.9455 - val_loss: 1.5644 - val_accuracy: 0.7470
Epoch 73/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1602 - accuracy:
0.9510 - val_loss: 0.8407 - val_accuracy: 0.8130
Epoch 74/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1241 - accuracy:
0.9560 - val_loss: 1.0173 - val_accuracy: 0.7940
Epoch 75/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1330 - accuracy:
0.9510 - val_loss: 0.9791 - val_accuracy: 0.7960
Epoch 76/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1409 - accuracy:
0.9530 - val_loss: 0.9169 - val_accuracy: 0.8030
Epoch 77/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1438 - accuracy:
0.9470 - val_loss: 1.2979 - val_accuracy: 0.7740
Epoch 78/100
63/63 [==============================] - 5s 74ms/step - loss: 0.1460 - accuracy:
0.9575 - val_loss: 1.3045 - val_accuracy: 0.8070
Epoch 79/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1460 - accuracy:
0.9500 - val_loss: 1.5664 - val_accuracy: 0.7800
Epoch 80/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1294 - accuracy:
0.9625 - val_loss: 0.9682 - val_accuracy: 0.8250
Epoch 81/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1608 - accuracy:
0.9530 - val_loss: 1.1262 - val_accuracy: 0.8150
Epoch 82/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1496 - accuracy:
0.9475 - val_loss: 0.8831 - val_accuracy: 0.8120
Epoch 83/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1541 - accuracy:
0.9460 - val_loss: 1.2917 - val_accuracy: 0.7740
Epoch 84/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1155 - accuracy:
0.9580 - val_loss: 1.0644 - val_accuracy: 0.7940
Epoch 85/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1365 - accuracy:
0.9600 - val_loss: 1.3014 - val_accuracy: 0.8090
Epoch 86/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1360 - accuracy:
0.9530 - val_loss: 1.2138 - val_accuracy: 0.8090
```

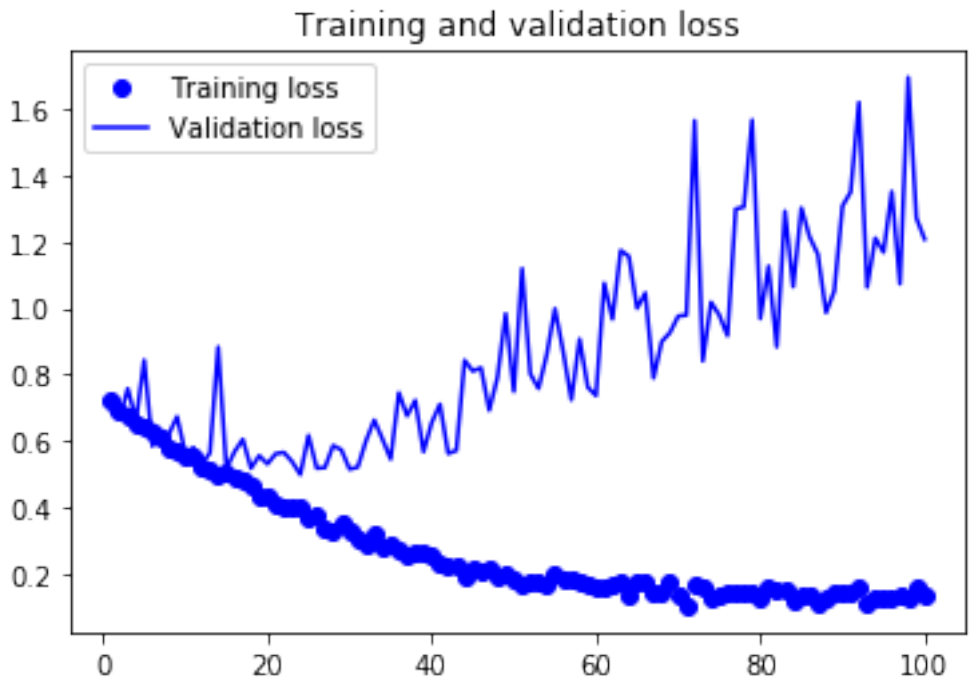
```
Epoch 87/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1110 - accuracy:
0.9630 - val_loss: 1.1623 - val_accuracy: 0.8020
Epoch 88/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1234 - accuracy:
0.9580 - val_loss: 0.9864 - val_accuracy: 0.8130
Epoch 89/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1461 - accuracy:
0.9560 - val_loss: 1.0514 - val_accuracy: 0.7910
Epoch 90/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1416 - accuracy:
0.9490 - val_loss: 1.3073 - val_accuracy: 0.8000
Epoch 91/100
63/63 [==============================] - 5s 75ms/step - loss: 0.1430 - accuracy:
0.9575 - val_loss: 1.3496 - val_accuracy: 0.7940
Epoch 92/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1562 - accuracy:
0.9515 - val_loss: 1.6192 - val_accuracy: 0.7570
Epoch 93/100
63/63 [==============================] - 5s 78ms/step - loss: 0.1145 - accuracy:
0.9630 - val_loss: 1.0627 - val_accuracy: 0.8040
Epoch 94/100
63/63 [==============================] - 5s 77ms/step - loss: 0.1268 - accuracy:
0.9590 - val_loss: 1.2107 - val_accuracy: 0.8170
Epoch 95/100
63/63 [==============================] - 5s 78ms/step - loss: 0.1292 - accuracy:
0.9615 - val_loss: 1.1680 - val_accuracy: 0.8270
Epoch 96/100
63/63 [==============================] - 5s 77ms/step - loss: 0.1251 - accuracy:
0.9595 - val_loss: 1.3509 - val_accuracy: 0.8140
Epoch 97/100
63/63 [==============================] - 5s 78ms/step - loss: 0.1341 - accuracy:
0.9575 - val_loss: 1.0727 - val_accuracy: 0.8200
Epoch 98/100
63/63 [==============================] - 5s 76ms/step - loss: 0.1240 - accuracy:
0.9615 - val_loss: 1.6949 - val_accuracy: 0.7930
Epoch 99/100
63/63 [==============================] - 5s 77ms/step - loss: 0.1630 - accuracy:
0.9530 - val_loss: 1.2710 - val_accuracy: 0.8360
Epoch 100/100
63/63 [==============================] - 5s 77ms/step - loss: 0.1384 - accuracy:
0.9585 - val_loss: 1.2073 - val_accuracy: 0.8320
```

2.3 Plotting the training results

- Overfitting appears after 30 epochs

[18]:
```
accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





2.4 Evaluation on the test dataset

- Accuracy: about 78%
- Without image augmentation: about 68%

[19]:
```
test_model = keras.models.load_model(
    "convnet_image_augmented.h5")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

```
63/63 [==============================] - 1s 11ms/step - loss: 0.5149 - accuracy:
0.7810
Test accuracy: 0.781
```